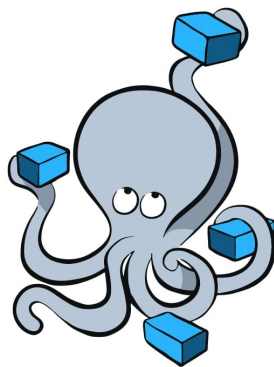


쉬운 도커

PART9. Docker Compose

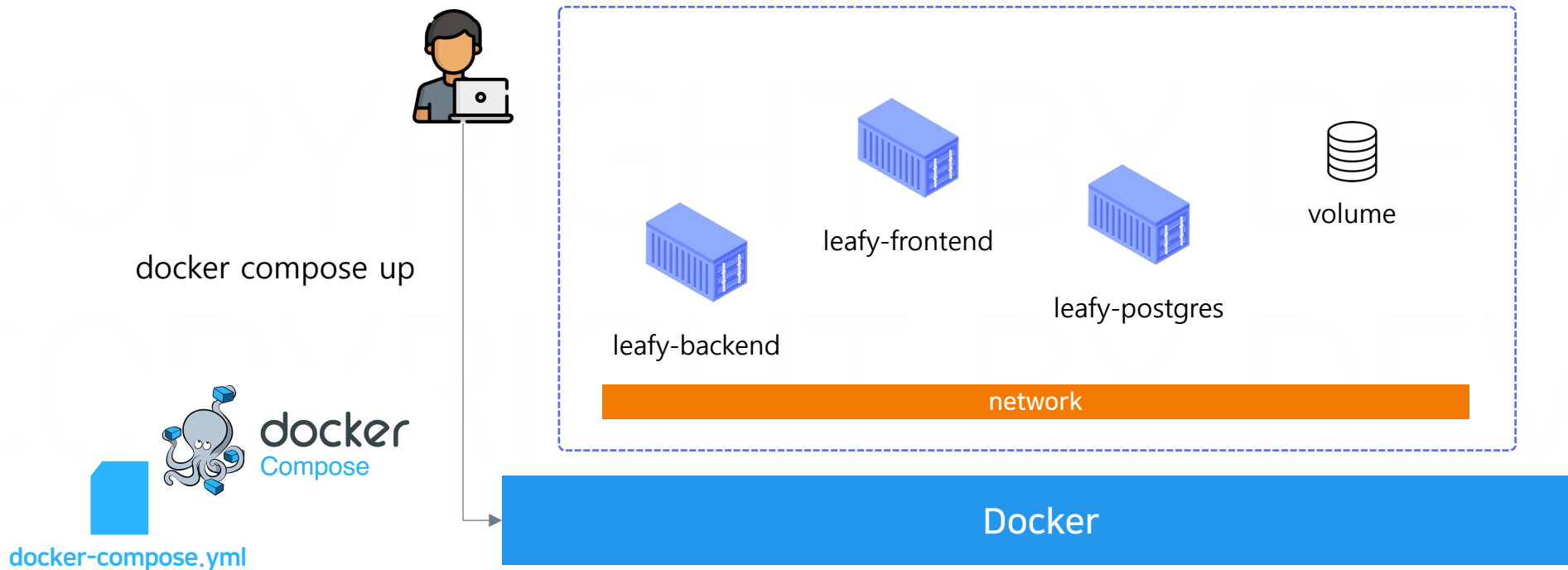


docker
Compose

PART9. Docker Compose

Docker Compose 기본

- 도커 컴포즈는 여러 개의 Docker 컨테이너들을 관리하는 도구입니다.
- 도커 컴포즈는 도커 데스크탑 설치 시 기본으로 설치됩니다.
- 한 번의 명령어로 여러 개의 컨테이너를 한번에 실행하거나 종료할 수 있습니다.
- 로컬 개발 환경에서 활용하기 편리합니다.



- 도커 컴포즈를 통해 관리할 컨테이너(서비스)를 docker-compose.yml 파일에 정의합니다.
- yml은 JSON과 같이 데이터를 작성하기 위한 양식입니다. JSON과 유사하지만 간결하고 가독성이 더 뛰어납니다.
- JSON은 {}와 "" , 가 필수인 반면 YAML은 띄어쓰기 기반으로 정보를 구분합니다.

JSON 포맷 예시

```
{
  "name": "John Doe",
  "age": 30,
  "isAlive": true,
  "hobbies": [
    "reading",
    "gaming",
    "coding"
  ],
  "address": {
    "street": "123 Main St",
    "city": "Anytown",
    "state": "NY"
  }
}
```

YAML 포맷 예시

```
name: John Doe
age: 30
isAlive: true
hobbies:
  - reading
  - gaming
  - coding
address:
  street: 123 Main St
  city: Anytown
  state: NY
```

docker compose up -d

YAML 파일에 정의된 서비스 생성 및 시작

docker compose ps

현재 실행중인 서비스 상태 표시

docker compose build

현재 실행중인 서비스의 이미지만 빌드

docker compose logs

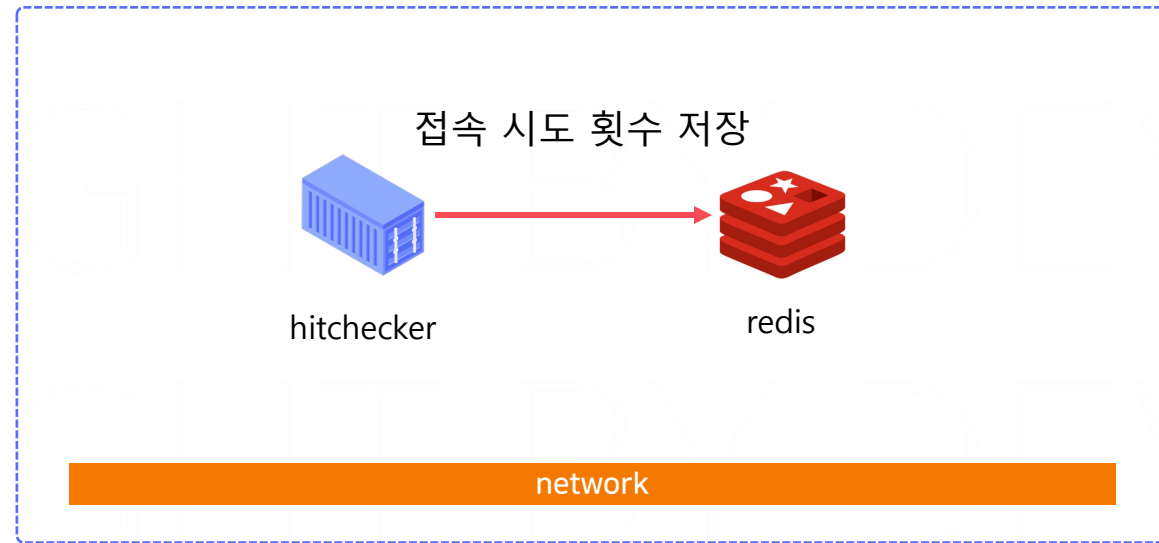
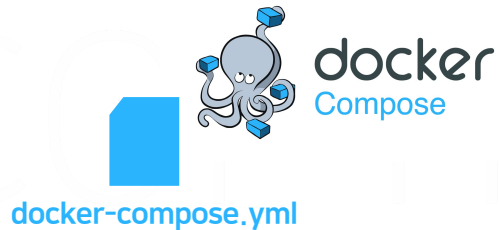
실행 중인 서비스의 로그 표시

docker compose down

YAML 파일에 정의된 서비스 종료 및 제거

- hitchecker는 접속 시도 횟수를 외부 캐시 저장소인 Redis에 저장합니다.
- hitchecker가 종료 또는 재생성 되어도 Redis에 접속 시도 회수가 저장되어 있습니다.
- docker compose를 사용해서 한 번에 hitchecker 와 Redis 서버를 구성할 수 있습니다.

docker compose up



Docker

easydocker/build/hitchecker/app/app.py

```
from flask import Flask
from redis import Redis

app = Flask(__name__)
redis = Redis(host='redis', port=6379) ----- 레디스 접속 정보 설정

@app.route('/')
def hello():
    redis.incr('hits') ----- / 경로 접근 시 redis의 캐시 값을 증가시키고 증가된 값을 출력
    return 'Hit Checked! 이 페이지는 %s 번 방문되었습니다.' % redis.get('hits').decode('utf-8')

if __name__ == "__main__":
    app.run(host="0.0.0.0", debug=True)
```

easydocker/build/hitchecker/app/Dockerfile

```
FROM python:3.7-alpine
WORKDIR /app
COPY . /app
RUN pip install flask redis ----- 애플리케이션 실행에 필요한 라이브러리 설치
CMD ["python", "app.py"]
```

easydocker/compose/hitchecker/docker-compose.yml

```
version: '3'
services:
  hitchecker:
    build: ./ ----- 이미지 빌드 시 사용할 Dockerfile 경로
    image: hitchecker:1.0.0 ----- 이미지 빌드 및 컨테이너 실행 시 사용할 이미지 태그
    ports:
      - "5000:5000"
    restart: always ----- 컨테이너 종료 시 자동으로 재시작
  redis:
    image: "redis:alpine" ----- 컨테이너 실행 시 사용할 이미지 태그
```


Docker Compose 명령어 테스트

1. easydocker/compose/hitchecker 에서 실행, 이미지 빌드

```
$ docker compose build
```

2. 빌드된 이미지 확인

```
$ docker image ls
```

3. 컨테이너 실행

```
$ docker compose up -d
```

4. 컨테이너 상태 확인, localhost:5000 접속

```
$ docker compose ps
```

5. 컨테이너 로그 확인

```
$ docker compose logs
```

6. 컨테이너 삭제 후 재실행 및 localhost:5000 접속, 값이 그대로 남아있는지 확인

```
$ docker rm -f {컨테이너명}
```

7. 서비스의 삭제

```
$ docker compose down
```

```
version: '3'
```

```
services:
```

```
  hitchecker:
```

```
    build: ./app
```

```
    image: hitchecker:1.0.0
```

```
    ports:
```

```
      - "5000:5000"
```

```
  redis:
```

```
    image: "redis:alpine"
```

API 버전을 정의

실행할 서비스들을 정의

hitchecker 컨테이너의 경우 사용자가 개발한 애플리케이션이기 때문에 실행 시 빌드 필요

애플리케이션 실행 시 포트 지정

redis의 경우 외부의 이미지를 다운받아서 실행

07.hitchecker

app

 app.py

 Dockerfile

 docker-compose.yml

docker compose up -d --build

로컬에 이미지가 있어도 다시 이미지를 빌드

easydocker/build/multidb/docker-compose.yml

```
version: '3'
services:
  postgres-primary-0:
    image: bitnami/postgresql-repmgr:15
    volumes:
      - postgres_primary_data:/bitnami/postgresql
    environment:
      POSTGRESQL_POSTGRES_PASSWORD: adminpassword
      POSTGRESQL_USERNAME: myuser
      POSTGRESQL_PASSWORD: mypassword
      POSTGRESQL_DATABASE: mydb
      REPMGR_PASSWORD: repmgrpassword
      REPMGR_PRIMARY_HOST: postgres-primary-0
      REPMGR_PRIMARY_PORT: 5432
      REPMGR_PARTNER_NODES: postgres-primary-0,postgres-standby-
1:5432
      REPMGR_NODE_NAME: postgres-primary-0
      REPMGR_NODE_NETWORK_NAME: postgres-primary-0
      REPMGR_PORT_NUMBER: 5432
```

```
postgres-standby-1:
  image: bitnami/postgresql-repmgr:15
  volumes:
    - postgres_standby_data:/bitnami/postgresql
  environment:
    POSTGRESQL_POSTGRES_PASSWORD: adminpassword
    POSTGRESQL_USERNAME: myuser
    POSTGRESQL_PASSWORD: mypassword
    POSTGRESQL_DATABASE: mydb
    REPMGR_PASSWORD: repmgrpassword
    REPMGR_PRIMARY_HOST: postgres-primary-0
    REPMGR_PRIMARY_PORT: 5432
    REPMGR_PARTNER_NODES: postgres-primary-0,postgres-standby-1:5432
    REPMGR_NODE_NAME: postgres-standby-1
    REPMGR_NODE_NETWORK_NAME: postgres-standby-1
    REPMGR_PORT_NUMBER: 5432

volumes:
  postgres_primary_data:
  postgres_standby_data:
```

easydocker/build/multidb-env/docker-compose.yml

```
version: '3'
x-environment: &common_environment
  POSTGRESQL_POSTGRES_PASSWORD: adminpassword
  POSTGRESQL_USERNAME: myuser
  POSTGRESQL_PASSWORD: mypassword
  POSTGRESQL_DATABASE: mydb
  REPMGR_PASSWORD: repmgrpassword
  REPMGR_PRIMARY_HOST: postgres-primary-0
  REPMGR_PRIMARY_PORT: 5432
  REPMGR_PORT_NUMBER: 5432

services:
  postgres-primary-0:
    image: bitnami/postgresql-repmgr:15
    volumes:
      - postgres_primary_data:/bitnami/postgresql
    environment:
      <<: *common_environment
      REPMGR_PARTNER_NODES: postgres-primary-0,postgres-standby-1:5432
      REPMGR_NODE_NAME: postgres-primary-0
      REPMGR_NODE_NETWORK_NAME: postgres-primary-0
```

```
postgres-standby-1:
  image: bitnami/postgresql-repmgr:15
  volumes:
    - postgres_standby_data:/bitnami/postgresql
  environment:
    <<: *common_environment
    REPMGR_PARTNER_NODES: postgres-primary-0,postgres-standby-1:5432
    REPMGR_NODE_NAME: postgres-standby-1
    REPMGR_NODE_NETWORK_NAME: postgres-standby-1

volumes:
  postgres_primary_data:
  postgres_standby_data:
```

PART9. Docker Compose

Leafy Docker Compose

- 도커 컴포즈를 통해 Leafy 애플리케이션을 개발자 PC에 빠르게 구성할 수 있습니다.
- Leafy front 개발자 또는 Leafy Backend 개발자는 나머지 모듈 들을 빠르게 구성할 수 있습니다.

2.docker compose로 전체 환경 빠르게 구성, 테스트 진행

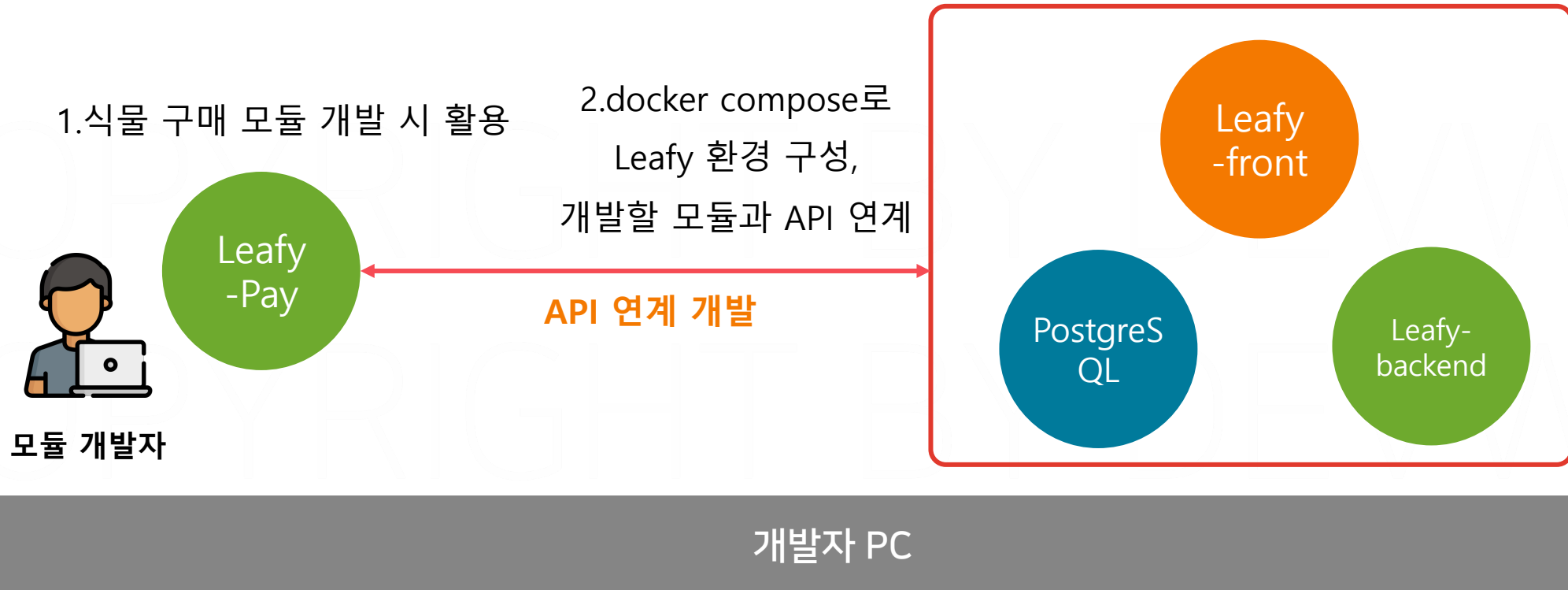


1.프론트엔드 소스코드 수정



개발자 PC

- 도커 컴포즈를 통해 Leafy 애플리케이션을 개발자 PC에 빠르게 구성할 수 있습니다.
- Leafy front 개발자 또는 Leafy Backend 개발자는 나머지 모듈 들을 빠르게 구성할 수 있습니다.
- 외부 모듈을 개발하는 개발자는 PC에 Leafy-front, Leafy-backend 소스코드가 없어도 빠르게 환경을 구동시킬 수 있습니다.



디렉터리 이동 및 소스코드 상태 변경

1. easydocker/leafy, 소스코드 작성을 뛰어넘으려면 git switch 07-compose 실행

```
git reset --hard HEAD
```

```
git switch 07-compose-init
```

easydocker/leafy/docker-compose.yml

```
version: '3'
```

```
services:
```

```
  leafy-postgres:
```

```
    image: devwikirepo/leafy-postgres:1.0.0
```

```
  leafy-backend:
```

```
    build: ./leafy-backend
```

```
    image: leafy-backend:5.0.0-compose
```

```
    environment:
```

```
      - DB_URL=leafy-postgres
```

```
    depends_on:
```

```
      - leafy-postgres
```

```
  leafy-front:
```

```
    build: ./leafy-frontend
```

```
    image: leafy-front:5.0.0-compose
```

```
    environment:
```

```
      - BACKEND_HOST=leafy-backend
```

```
    ports:
```

```
      - 80:80
```

```
    depends_on:
```

```
      - leafy-backend
```

leafy-postgres, leafy-backend, leafy-front
3개의 컨테이너 관리

```
docker run --name leafy-postgres --network leafy-  
network devwikirepo/leafy-postgres:1.0.0
```

docker compose 실행 시 컨테이너들은
모두 같은 네트워크에 속하게 구성됨

easydocker/leafy/docker-compose.yml

```
version: '3'
services:
  leafy-postgres:
    image: devwikirepo/leafy-postgres:1.0.0
  leafy-backend:
    build: ./leafy-backend
    image: leafy-backend:5.0.0-compose
    environment:
      - DB_URL=leafy-postgres
    depends_on:
      - leafy-postgres
  leafy-front:
    build: ./leafy-frontend
    image: leafy-front:5.0.0-compose
    environment:
      - BACKEND_HOST=leafy-backend
    ports:
      - 80:80
    depends_on:
      - leafy-backend
```

leafy-postgres, leafy-backend, leafy-front
3개의 컨테이너 관리

▼ LEAFY

- > leafy-backend
- > leafy-frontend
- > leafy-postgresql
- 🔗 docker-compose.yml

leafy-backend:5.0.0-compose 이미지가 있으면 그대로 실행,
없으면 ./leafy-backend 폴더의 이미지 빌드

`docker build -t leafy-backend:5.0.0-compose ./leafy-backend`

`docker run -e DB_URL=leafy-postgres --name leafy-backend
--network leafy-network devwikirepo/leafy-backend:1.0.0`

easydocker/leafy/docker-compose.yml

version: '3'

services:

leafy-postgres, leafy-backend, leafy-front
3개의 컨테이너 관리

leafy-postgres:

image: devwikirepo/leafy-postgres:1.0.0

leafy-backend:

build: ./leafy-backend

image: leafy-backend:5.0.0-compose

environment:

- DB_URL=leafy-postgres

Database 접속을 위한 URL 환경 변수 설정
DB_URL = DB컨테이너명 = leafy-postgres

depends_on:

- leafy-postgres

leafy-front:

build: ./leafy-frontend

image: leafy-front:5.0.0-compose

environment:

- BACKEND_HOST=leafy-backend

ports:

- 80:80

depends_on:

- leafy-backend

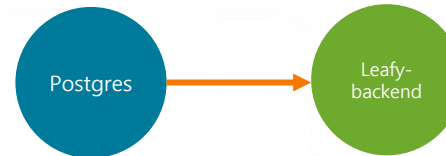
`docker run -e DB_URL=leafy-postgres --name leafy-backend
--network leafy-network devwikirepo/leafy-backend:1.0.0`

easydocker/leafy/docker-compose.yml

```
version: '3'
services:
  leafy-postgres:
    image: devwikirepo/leafy-postgres:1.0.0
  leafy-backend:
    build: ./leafy-backend
    image: leafy-backend:5.0.0-compose
    environment:
      - DB_URL=leafy-postgres
    depends_on:
      - leafy-postgres
  leafy-front:
    build: ./leafy-frontend
    image: leafy-front:5.0.0-compose
    environment:
      - BACKEND_HOST=leafy-backend
    ports:
      - 80:80
    depends_on:
      - leafy-backend
```

leafy-postgres, leafy-backend, leafy-front
3개의 컨테이너 관리

`depends_on` 은 특정 컨테이너가 실행될 때 까지 컨테이너 실행을 보류합니다.
leafy-postgres 컨테이너가 실행되기 전까지 leafy-backend 컨테이너는
실행되지 않습니다.



실행 대기

`depends_on` 을 정의하지 않으면 모든 컨테이너가 동시에 실행됩니다.

`depends_on` 을 정의해도 컨테이너가 Running 상태에서 내부 프로그램
실행에 시간이 오래 걸리는 경우 문제가 발생할 수 있습니다.

easydocker/leafy/docker-compose.yml

```
version: '3'
services:
  leafy-postgres:
    image: devwikirepo/leafy-postgres:1.0.0
  leafy-backend:
    build: ./leafy-backend
    image: leafy-backend:5.0.0-compose
    environment:
      - DB_URL=leafy-postgres
    depends_on:
      - leafy-postgres
  leafy-front:
    build: ./leafy-frontend
    image: leafy-front:5.0.0-compose
    environment:
      - BACKEND_HOST=leafy-backend
    ports:
      - 80:80
    depends_on:
      - leafy-backend
```

leafy-postgres, leafy-backend, leafy-front
3개의 컨테이너 관리

▼ LEAFY

> leafy-backend

> leafy-frontend

> leafy-postgresql

 docker-compose.yml

```
docker build -t leafy-front:5.0.0-compose ./leafy-frontend
```

```
docker run -e BACKEND_HOST=leafy-backend -p 80:80
--name leafy-front --network leafy-network leafy-
front:5.0.0-compose
```

ports 는 컨테이너의 포트포워딩을 정의합니다.

Docker Compose 실행 테스트

1. 컨테이너 실행 및 서비스 접속 테스트 후 컨테이너 종료

```
$ docker compose up -d
```

```
$ docker compose down
```

easydocker/leafy/docker-compose.yml

```
version: '3'
services:
  leafy-postgres:
    build: ./leafy-postgresql
    image: leafy-postgres:5.0.0-compose
    volumes:
      - mydata:/var/lib/postgresql/data
```

개발 시 소스 변경 및 반영이 필요할 경우 build 옵션을 추가합니다.

volume을 추가합니다.

```
leafy-backend:
  build: ./leafy-backend
  image: leafy-backend:5.0.0-compose
  environment:
    - DB_URL=leafy-postgres
  depends_on:
    - leafy-postgres
```

```
leafy-front:
  build: ./leafy-frontend
  image: leafy-front:5.0.0-compose
  environment:
    - BACKEND_HOST=leafy-backend
  ports:
    - 80:80
  depends_on:
    - leafy-backend
```

```
volumes:
  mydata:
```

volume 정의

▼ LEAFY

- > leafy-backend
- > leafy-frontend
- > leafy-postgresql
-  docker-compose.yml

```
docker run -d --name leafy-postgres -v
mydata:/var/lib/postgresql/data --network leafy-
network devwikirepo/leafy-postgres:1.0.0
```

easydocker/leafy/docker-compose.yml

```
version: '3'
services:
  leafy-postgres:
    build: ./leafy-postgresql
    image: leafy-postgres:5.0.0-compose
    volumes:
      - mydata:/var/lib/postgresql/data
```

```
    deploy:
      resources:
        limits:
          cpus: '1'
          memory: 256M
      restart: always
```

```
  leafy-backend:
    build: ./leafy-backend
    image: leafy-backend:5.0.0-compose
    environment:
      - DB_URL=leafy-postgres
    depends_on:
      - leafy-postgres
```

```
    deploy:
      resources:
        limits:
          cpus: '1.5'
          memory: 512M
      restart: on-failure
```

```
  leafy-front:
    build: ./leafy-frontend
    image: leafy-front:5.0.0-compose
    environment:
      - BACKEND_HOST=leafy-backend
    ports:
      - 80:80
    depends_on:
      - leafy-backend
```

```
    deploy:
      resources:
        limits:
          cpus: '0.5'
          memory: 64M
      restart: on-failure
```

```
volumes:
  mydata:
```

deploy:
resources:
limits:
cpus: '최대CPU사용량'
memory: 최대RAM사용량

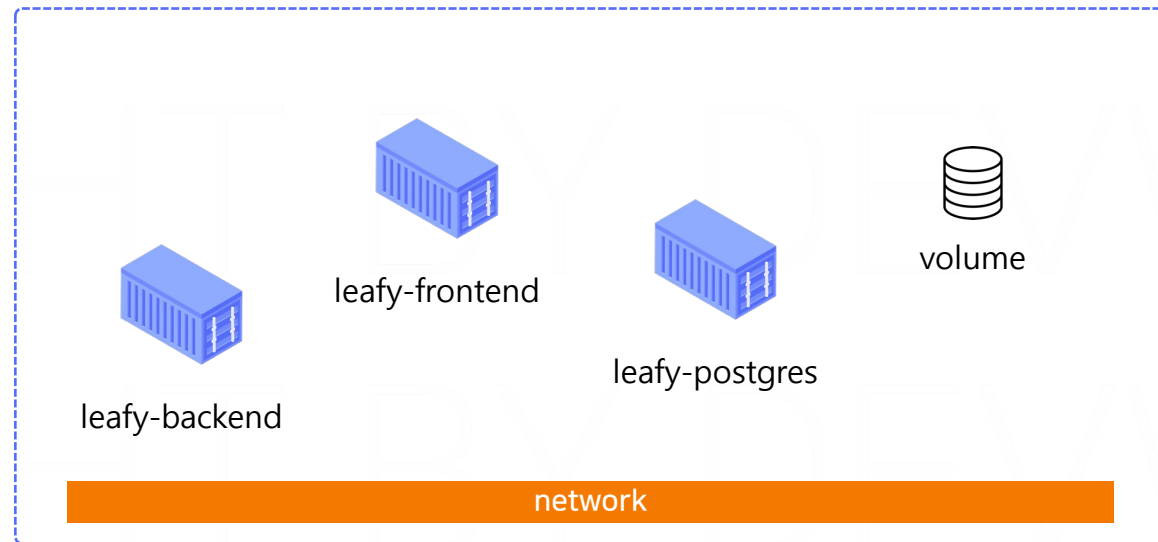
restart:
on-failure: 실패 시에만 재실행
always: 무조건 재실행

easydocker/leafy/docker-compose.yml

```
version: '3'
services:
  leafy-postgres:
  ...
  leafy-backend:
  ...
  leafy-front:
  ...
volumes:
  mydata:
```



docker compose up



Docker